

mostly asked question on spring boot in java intrevwer with details answer and also provide soe questions from comapiesnes

Perfect 👍 — let's go through **most commonly asked Spring Boot interview questions** (with **detailed answers**) and then I'll include **real company-level questions** from top firms like **TCS, Infosys, Cognizant, Capgemini, JP Morgan, and Amazon**.

🧩 Top Spring Boot Interview Questions (with Detailed Answers)

1. What is Spring Boot? Why do we use it?

Answer:

Spring Boot is a framework built on top of the Spring Framework that simplifies application development by eliminating the need for complex configuration.

Key Features:

- **Auto-Configuration:** Automatically configures beans based on dependencies.
- **Starter Dependencies:** Predefined dependencies for specific use-cases (`spring-boot-starter-web` , `spring-boot-starter-data-jpa`).
- **Embedded Server:** Comes with Tomcat/Jetty/Undertow.
- **Actuator:** Provides production-ready features like health checks and metrics.

Example:

Instead of configuring `dispatcher-servlet.xml` , you just add:

```
java
```

 Copy code

```
@SpringBootApplication
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }
}
```

2. What are Spring Boot Starters?

Answer:

Starters are a set of dependency descriptors you can include in your project to get a particular feature.

Example:

If you add:

xml

 Copy code

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

👉 It includes dependencies like `spring-web`, `spring-webmvc`, and `tomcat`.

3. What is Spring Boot Auto-Configuration?

Answer:

Auto-configuration automatically sets up beans based on what's available in the classpath.

Example:

If `spring-webmvc` is on your classpath, Spring Boot automatically configures a `DispatcherServlet`.

You can disable it using:

java

 Copy code

```
@SpringBootApplication(exclude = {DataSourceAutoConfiguration.class})
```

4. What is @SpringBootApplication Annotation?

Answer:

It is a combination of:

- `@Configuration`
- `@EnableAutoConfiguration`
- `@ComponentScan`

This means Spring Boot:

1. Marks the class as configuration source.
2. Enables auto-configuration.
3. Scans the base package for components.

5. What is the difference between `@Component`, `@Service`, and `@Repository`?**Answer:**

All three mark a class as a Spring-managed bean.

Annotation	Purpose
<code>@Component</code>	Generic bean
<code>@Service</code>	Business logic layer
<code>@Repository</code>	DAO layer — adds translation for database exceptions

6. How does Spring Boot handle database connections?**Answer:**

Spring Boot auto-configures a `DataSource` when database-related dependencies are found.

Example configuration (`application.properties`):

```
properties
```

[Copy code](#)

```
spring.datasource.url=jdbc:mysql://localhost:3306/demo
spring.datasource.username=root
```

```
spring.datasource.password=root  
spring.jpa.hibernate.ddl-auto=update
```

7. What is Spring Boot Actuator?

Answer:

Actuator provides production-ready endpoints for monitoring your app.

Examples:

- /actuator/health
- /actuator/metrics
- /actuator/env

To enable:

xml

 Copy code

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-actuator</artifactId>  
</dependency>
```

8. How do you handle exceptions in Spring Boot?

Answer:

Use `@ControllerAdvice` and `@ExceptionHandler`.

java

 Copy code

```
@ControllerAdvice  
public class GlobalExceptionHandler {  
    @ExceptionHandler(Exception.class)  
    public ResponseEntity<String> handleException(Exception e) {  
        return new ResponseEntity<>("Error: " + e.getMessage(), HttpStatus.INTERNAL_SERVER_ERROR)  
    }  
}
```

9. What is the use of application.properties / application.yml?

Answer:

Used for configuring:

- Server port
- Database
- Profiles
- Logging

Example:

properties

 Copy code

```
server.port=8081
spring.application.name=myapp
```

10. What are Spring Profiles?

Answer:

Profiles allow you to create environment-specific configurations (dev, test, prod).

Example:

java

 Copy code

```
@Profile("dev")
@Component
public class DevConfig {}
```

Run with:

bash

 Copy code

```
java -jar app.jar --spring.profiles.active=dev
```

11. What is @RestController vs @Controller?

Answer:

Annotation	Purpose
@Controller	Used to return view (HTML/JSP)
@RestController	Combines @Controller and @ResponseBody — returns JSON/XML

12. What is the difference between Spring Boot and Spring MVC?

Answer:

Feature	Spring MVC	Spring Boot
Setup	Manual XML config	Auto-configured
Dependencies	Manual	Starters
Server	External	Embedded
Focus	Web framework	Rapid development

13. How to secure a Spring Boot application?

Answer:

Use `spring-boot-starter-security`.

Example:

java

 Copy code

```
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {
    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http.authorizeRequests().anyRequest().authenticated()
            .and().httpBasic();
    }
}
```

```
}  
}
```

14. How can you create a custom starter in Spring Boot?

Answer:

Steps:

1. Create a new module.
 2. Add `@Configuration` class.
 3. Add `META-INF/spring.factories` file pointing to configuration class.
 4. Publish and use it as dependency.
-

15. How does Spring Boot integrate with AWS or cloud platforms?

Answer:

You can use:

- `spring-cloud-starter-aws` for S3, SNS, SQS integration.
 - Environment variables or secrets from Parameter Store.
 - Deploy as Lambda using Spring Cloud Function.
-



Company-Level Spring Boot Interview Questions

TCS / Infosys / Wipro

1. What is the difference between Spring Boot and Spring MVC?
 2. How does Spring Boot manage dependency injection automatically?
 3. Explain the flow of a REST API call in Spring Boot.
 4. What is `CommandLineRunner` and its use?
 5. How do you connect Spring Boot with MySQL or Oracle?
-

Cognizant / Capgemini

1. Explain auto-configuration in depth — how does it work internally?

2. What is the use of `@EnableAutoConfiguration` and `spring.factories` ?
 3. How does Spring Boot handle circular dependencies?
 4. How to define multiple profiles and switch between them?
 5. How does Spring Boot handle transaction management?
-

JP Morgan / Barclays

1. What happens internally when you run a Spring Boot app using `SpringApplication.run()` ?
 2. How to handle large file uploads in Spring Boot?
 3. How to configure async processing using `@Async` ?
 4. What are different ways to externalize configuration in Spring Boot?
 5. Explain distributed tracing with Spring Cloud Sleuth.
-

Amazon / Microsoft

1. How does Spring Boot achieve inversion of control and dependency injection?
 2. How would you design a microservice using Spring Boot and AWS Lambda?
 3. What is Spring Cloud Config and how does it manage configuration across microservices?
 4. How to implement rate limiting or circuit breaker in Spring Boot?
 5. How would you secure REST APIs using JWT in Spring Boot?
-

Would you like me to also include **Spring Boot + Microservices interview questions** (like Eureka, Feign Client, Config Server, Gateway, Circuit Breaker) next? That's usually asked after Spring Boot basics.



mostly asked question on exceptions ahndling

Excellent 🍌 — exception handling is one of the **most frequently asked topics** in Spring Boot interviews (and even in core Java + Spring rounds).

Let's go step by step — covering **theory + practical examples + company-level questions** 📌

🌸 Spring Boot Exception Handling — Interview Questions with Detailed Answers

1. What is exception handling in Spring Boot?

Answer:

Exception handling in Spring Boot allows developers to manage and customize how exceptions are handled in a REST API or application.

Spring provides mechanisms to:

- Catch and process exceptions globally.
- Return meaningful error responses.
- Maintain clean controller code.

2. What are the different ways to handle exceptions in Spring Boot?

Answer:

Spring Boot provides **four main ways**:

Approach	Description	Example
Try-Catch Block	Handle exceptions inside controller/service manually	<code>try { } catch(Exception e) {}</code>
<code>@ExceptionHandler</code>	Handle specific exceptions at controller level	<code>@ExceptionHandler(ResourceNotFoundException.class)</code>
<code>@ControllerAdvice</code>	Global exception handling across all controllers	<code>@ControllerAdvice</code>

Approach	Description	Example
<code>ResponseStatusException / @ResponseStatus</code>	Map exceptions to HTTP status codes	throw new <code>ResponseStatusException(HttpStatus.NOT_FOUND, "User not found")</code>

3. What is `@ExceptionHandler` annotation used for?

Answer:

It is used inside a controller (or `@ControllerAdvice`) to handle specific exceptions.

Example:

java

 Copy code

```
@RestController
public class UserController {

    @GetMapping("/user/{id}")
    public User getUser(@PathVariable Long id) {
        if(id == 0) throw new ResourceNotFoundException("User not found");
        return new User(id, "John");
    }

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<String> handleResourceNotFound(ResourceNotFoundException ex) {
        return new ResponseEntity<>(ex.getMessage(), HttpStatus.NOT_FOUND);
    }
}
```

4. What is `@ControllerAdvice` and how is it different from `@ExceptionHandler`?

Answer:

Feature	@ExceptionHandler	@ControllerAdvice
Scope	Controller-specific	Global (applies to all controllers)
Location	Inside controller class	Separate global class
Usage	Handle exceptions locally	Centralized error handling

Example:

java

 Copy code

```

@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<String> handleResourceNotFound(ResourceNotFoundException ex) {
        return new ResponseEntity<>(ex.getMessage(), HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<String> handleGeneric(Exception ex) {
        return new ResponseEntity<>("Something went wrong!", HttpStatus.INTERNAL_SERVER_ERROR);
    }
}

```

5. What is the use of @ResponseStatus annotation?**Answer:**

Used to mark a custom exception with a specific HTTP status code.

Example:

java

 Copy code

```

@ResponseStatus(HttpStatus.NOT_FOUND)
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}

```

```
}  
}
```

If this exception is thrown, Spring automatically returns **HTTP 404**.

6. What is `ResponseStatusException` and when to use it?

Answer:

Introduced in Spring 5, it allows throwing exceptions with an HTTP status directly without creating a custom exception.

Example:

java

 Copy code

```
@GetMapping("/user/{id}")  
public User getUser(@PathVariable Long id) {  
    if (id == 0) {  
        throw new ResponseStatusException(HttpStatus.NOT_FOUND, "User not found")  
    }  
    return new User(id, "John");  
}
```

7. What is the difference between `@ResponseStatus` and `ResponseStatusException` ?

Feature	@ResponseStatus	ResponseStatusException
Type	Annotation	Class
Use Case	Static mapping of exception to status	Dynamic handling in runtime
Example	On custom exception class	Thrown directly inside code

8. How to create a custom error response structure?

Answer:

Define a class to represent the response:

java

 Copy code

```
public class ErrorResponse {  
    private String message;  
    private String details;  
    private LocalDateTime timestamp;  
  
    // constructor, getters, setters  
}
```

Use it in global handler:

java

 Copy code

```
@ExceptionHandler(Exception.class)  
public ResponseEntity<ErrorResponse> handleAllExceptions(Exception ex) {  
    ErrorResponse error = new ErrorResponse(ex.getMessage(), "Server Error", LocalD  
    return new ResponseEntity<>(error, HttpStatus.INTERNAL_SERVER_ERROR);  
}
```

9. How do you handle validation exceptions (@Valid , @Validated) in Spring Boot?

Answer:

When using validation, invalid input throws `MethodArgumentNotValidException` .

Handle it globally:

java

 Copy code

```
@ExceptionHandler(MethodArgumentNotValidException.class)  
public ResponseEntity<ErrorResponse> handleValidationErrors(MethodArgumentNotValid  
    String errorMsg = ex.getBindingResult().getAllErrors().get(0).getDefaultMessag  
    ErrorResponse error = new ErrorResponse(errorMsg, "Validation Failed", LocalD
```

```
return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);  
}
```

10. How can you handle exceptions for REST API and return JSON?

Answer:

Use `@RestControllerAdvice` instead of `@ControllerAdvice` — it automatically returns JSON instead of a view.

java

 Copy code

```
@RestControllerAdvice  
public class RestExceptionHandler {  
  
    @ExceptionHandler(ResourceNotFoundException.class)  
    public ResponseEntity<Map<String, String>> handleNotFound(ResourceNotFoundException ex)  
    {  
        Map<String, String> error = new HashMap<>();  
        error.put("error", ex.getMessage());  
        error.put("status", "404");  
        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);  
    }  
}
```



Company-Level Questions on Exception Handling

TCS / Infosys / Capgemini

1. What is the difference between `@ControllerAdvice` and `@ExceptionHandler` ?
2. How do you handle exceptions globally in Spring Boot?
3. What is the use of `@ResponseStatus` ?
4. How do you handle validation errors when using `@Valid` in Spring Boot REST APIs?
5. Can you return custom error messages from your exception handler?

Cognizant / JP Morgan / Barclays

1. How do you ensure that exception messages are consistent across microservices?
 2. Explain how `ResponseEntityExceptionHandler` works and how you can extend it.
 3. How can you log all exceptions globally without affecting controller logic?
 4. Can you explain how exception propagation works in Spring Boot?
 5. How to handle nested exceptions or root cause exceptions?
-

Amazon / Microsoft / PayPal

1. How would you design a unified error-handling mechanism for multiple microservices?
 2. What's the difference between checked and unchecked exceptions in a Spring Boot context?
 3. How do you handle exceptions in async methods (like `@Async`)?
 4. How do you map exceptions from service layer to meaningful REST responses?
 5. What are the best practices for exception handling in large-scale Spring Boot systems?
-

✓ Best Practices

- Always use **custom exception classes** for clarity.
 - Use `@RestControllerAdvice` for REST APIs.
 - Log exceptions in a **centralized** manner (with stack trace).
 - Use **meaningful HTTP codes** (404, 400, 500).
 - Avoid exposing internal error details to clients.
-

Would you like me to add a **small real-world exception handling structure** (like a ready-to-use `GlobalExceptionHandler.java` + custom `ErrorResponse` + controller example) next?

That's very useful for interview demos and projects.



please provide the answe for above compaines qiestions

Perfect ✅ — let's go through **detailed answers for all the company-level Spring Boot Exception Handling questions**, grouped by company (TCS, Infosys, Capgemini, Cognizant, JP Morgan, Barclays, Amazon, Microsoft, PayPal).

Each answer is practical and interview-ready, with **examples + reasoning** — just how interviewers expect you to explain.

TCS / Infosys / Capgemini — Basic to Intermediate

1. What is the difference between `@ControllerAdvice` and `@ExceptionHandler` ?

Answer:

- `@ExceptionHandler` handles exceptions **within a single controller**.
- `@ControllerAdvice` (Or `@RestControllerAdvice`) is a **global handler** for all controllers.

Example:

java

 Copy code

```
@ControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<String> handle(ResourceNotFoundException ex) {
        return new ResponseEntity<>(ex.getMessage(), HttpStatus.NOT_FOUND);
    }
}
```

👉 This ensures the same error-handling logic applies everywhere, keeping controller code clean.

2. How do you handle exceptions globally in Spring Boot?

Answer:

By using `@ControllerAdvice` Or `@RestControllerAdvice` .

It acts like a global interceptor of exceptions thrown from any controller.

Example:

java

 Copy code

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(Exception.class)
    public ResponseEntity<String> handleGlobalException(Exception ex) {
        return new ResponseEntity<>("Something went wrong: " + ex.getMessage(),
                                    HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

This avoids repetitive `try-catch` blocks in each controller.

3. What is the use of `@ResponseStatus` annotation?

Answer:

`@ResponseStatus` maps an exception to a specific HTTP status code **without writing extra handler code**.

Example:

java

 Copy code

```
@ResponseStatus(HttpStatus.NOT_FOUND)
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String msg) { super(msg); }
}
```

If this exception is thrown, Spring automatically returns **404 Not Found**.

4. How do you handle validation errors when using `@Valid` in Spring Boot REST APIs?

Answer:

Validation errors throw `MethodArgumentNotValidException`.

We handle it using `@ExceptionHandler`.

Example:

java

 Copy code

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Map<String, String>> handleValidationError(MethodArgumentNotValidException ex) {
    Map<String, String> errors = new HashMap<>();
    ex.getBindingResult().getFieldErrors().forEach(err ->
        errors.put(err.getField(), err.getDefaultMessage()));
    return new ResponseEntity<>(errors, HttpStatus.BAD_REQUEST);
}
```

This returns readable JSON for validation issues.

5. Can you return custom error messages from your exception handler?

Answer:

Yes, by creating a custom response object.

Example:

java

 Copy code

```
public class ErrorResponse {
    private String message;
    private LocalDateTime timestamp;
    private int status;
    // getters/setters
}

@ExceptionHandler(ResourceNotFoundException.class)
public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex)
```

```
ErrorResponse error = new ErrorResponse(ex.getMessage(), LocalDateTime.now()),
return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
}
```

Cognizant / JP Morgan / Barclays — Advanced Practical Questions

1. How do you ensure that exception messages are consistent across microservices?

Answer:

- Create a **common error response structure** (ErrorResponse DTO).
- Use **shared library or dependency** across services.
- Maintain **consistent error codes and message format**.

Example JSON:

json

 Copy code

```
{
  "errorCode": "USER_NOT_FOUND",
  "message": "Requested user does not exist",
  "timestamp": "2025-11-12T09:00:00Z"
}
```

All services follow this format to maintain uniformity.

2. Explain how ResponseEntityExceptionHandler works and how you can extend it.

Answer:

ResponseExceptionHandler is a base class provided by Spring that already handles many standard exceptions (e.g., HttpRequestMethodNotSupportedException ,
HttpMediaTypeNotSupportedException).

We can extend it to override or customize responses:

java

 Copy code

```

@RestControllerAdvice
public class CustomExceptionHandler extends ResponseEntityExceptionHandler {
    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(
        MethodArgumentNotValidException ex, HttpHeaders headers,
        HttpStatus status, WebRequest request) {

        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(err ->
            errors.put(err.getField(), err.getDefaultMessage()));

        return new ResponseEntity<>(errors, HttpStatus.BAD_REQUEST);
    }
}

```

3. How can you log all exceptions globally without affecting controller logic?

Answer:

Use `@ControllerAdvice` with logging frameworks like `SLF4J` or `Log4j2`.

java

 Copy code

```

@Slf4j
@RestControllerAdvice
public class GlobalLogger {
    @ExceptionHandler(Exception.class)
    public ResponseEntity<String> handle(Exception ex) {
        log.error("Exception occurred: ", ex);
        return new ResponseEntity<>("Internal Server Error", HttpStatus.INTERNAL_SERVER_ERROR);
    }
}

```

👉 Ensures consistent logging without modifying controller code.

4. Can you explain how exception propagation works in Spring Boot?

Answer:

When an exception occurs:

1. Spring checks for an applicable `@ExceptionHandler` in the same controller.
2. If not found, it looks in any `@ControllerAdvice`.
3. If still not found, it triggers default Spring Boot error handling (`BasicErrorController`) which returns `/error` JSON.

5. How to handle nested exceptions or root cause exceptions?

Answer:

Use `Throwable.getCause()` or Apache Commons `ExceptionUtils.getRootCause(ex)`.

Example:

java

 Copy code

```
@ExceptionHandler(Exception.class)
public ResponseEntity<String> handle(Exception ex) {
    Throwable rootCause = ExceptionUtils.getRootCause(ex);
    return new ResponseEntity<>("Root Cause: " + rootCause.getMessage(),
                                HttpStatus.INTERNAL_SERVER_ERROR);
}
```



Amazon / Microsoft / PayPal — System Design & Best Practices Level

1. How would you design a unified error-handling mechanism for multiple microservices?

Answer:

Steps:

1. Create a **shared error library** (common module) containing:

- ErrorResponse
- GlobalExceptionHandler
- CustomExceptions

2. Include it as a dependency in each microservice.
3. Use centralized logging (ELK / CloudWatch / Splunk).
4. Return uniform error structure and error codes.

This approach ensures consistency and reusability across services.

2. What's the difference between checked and unchecked exceptions in a Spring Boot context?

Answer:

Type	Inheritance	Must Handle?	Example	Spring Boot Handling
Checked	Extends Exception	Yes	IOException	Usually wrapped in RuntimeException
Unchecked	Extends RuntimeException	No	NullPointerException	Automatically propagated to @ExceptionHandler

Spring recommends **unchecked exceptions** for service and controller layers — they flow naturally to handlers.

3. How do you handle exceptions in async methods (@Async)?

Answer:

When using @Async , exceptions don't propagate to the main thread.

We can handle them using AsyncUncaughtExceptionHandler .

Example:

```
java
```

 Copy code

```
@Configuration
@EnableAsync
public class AsyncConfig implements AsyncConfigurer {
    @Override
    public AsyncUncaughtExceptionHandler getAsyncUncaughtExceptionHandler() {
        return (ex, method, params) ->
            System.out.println("Async error in method: " + method.getName() + " -
    }
}
```

4. How do you map exceptions from service layer to meaningful REST responses?

Answer:

Throw custom exceptions in service layer and handle them in `@ControllerAdvice`.

Example:

java

 Copy code

```
// Service
if (user == null)
    throw new ResourceNotFoundException("User not found");

// Global handler
@ExceptionHandler(ResourceNotFoundException.class)
public ResponseEntity<ErrorResponse> handle(ResourceNotFoundException ex) {
    return new ResponseEntity<>(new ErrorResponse(ex.getMessage(), LocalDateTime.
        HttpStatus.NOT_FOUND);
}
```

5. What are the best practices for exception handling in large-scale Spring Boot systems?

Answer: **Best Practices**

- Always use **custom exception classes** (e.g., `InvalidRequestException` , `UserNotFoundException`).
- Use `@RestControllerAdvice` for consistent JSON responses.